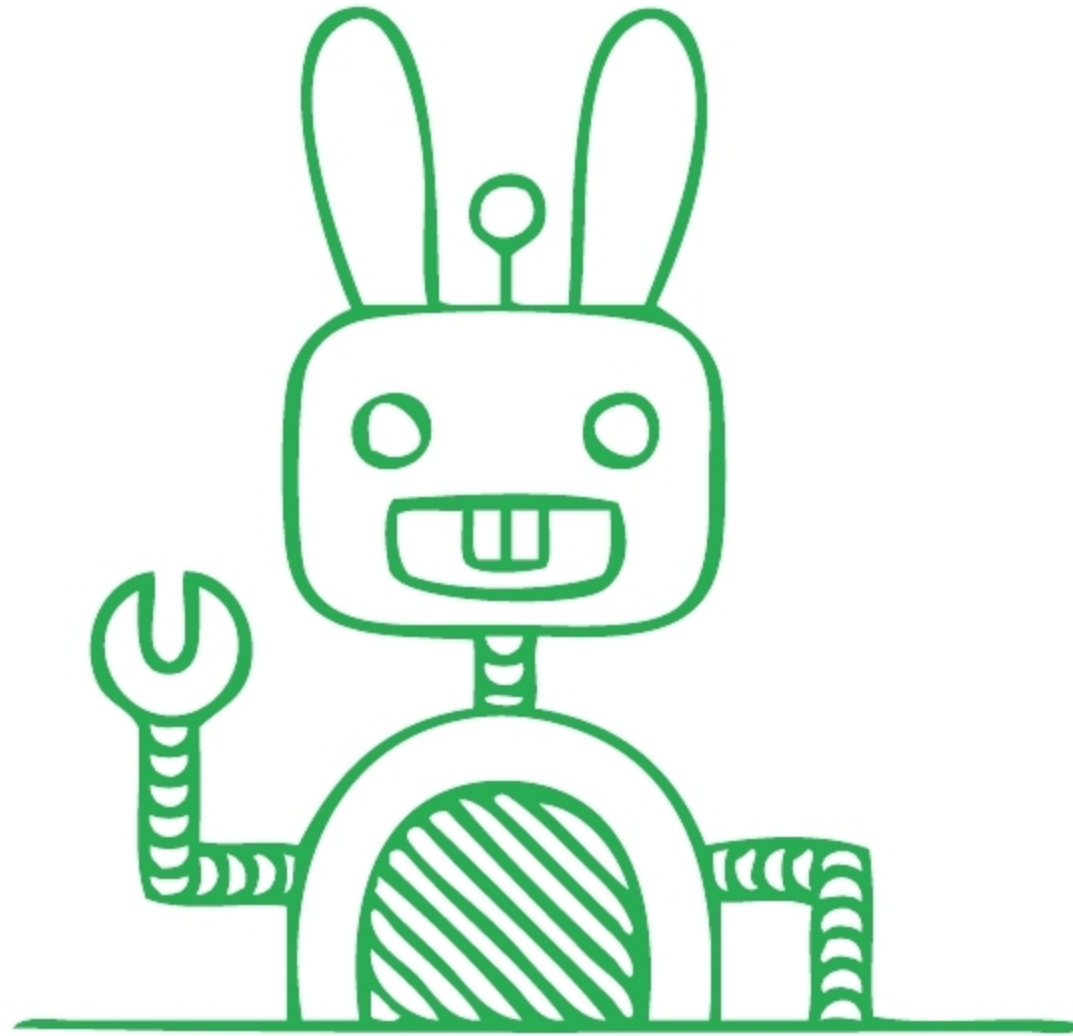


A Primer on RabbitMQ, the Popular Open Source Message Broker



RabbitMQ is the most popular and widely deployed message broker and it happens to be open source. Its versatility makes it easily deployable in the cloud and in situ. It supports high availability and multiple messaging protocols.

RabbitMQ is a message broker, i.e., software that manages and defines message queues. It is a queue manager that connects applications and transmits messages through them. A message can be data or information like a simple text message, or some important details about a task that should be initiated on some other application. Message queuing software stores all the messages until the destination system connects and takes the message out of the queue. The destination application then transmits the message in the proper manner.

The basic architecture of RabbitMQ has three main components.

- *Producers* – client applications that create messages and send them to

the broker/message queue.

- *Consumers* – connect to the queue and subscribe to the message that needs to be processed.
- *Broker* – a middleman who keeps messages in queue until the consumers take them.

Any software can be a producer, a consumer or both together. A message broker is used to reduce loads and hasten the delivery time of tasks. Tasks that normally take longer to process are forwarded to a third party to perform this job. This is faster as the only job for the third party is to process the task sent to it.

When to use RabbitMQ

Let's look at an example to understand this better. There is a website that allows

users to upload information, generates a PDF containing this information and emails this PDF back to the user. The tasks that take time involve handling of the data, making a PDF from the data and sending an email with the PDF attachment. This time can be minimised using a message queue (RabbitMQ).

Once the user has entered all the details, the application creates a new task called 'PDF processing' and puts in all the information, including the email address. This message is queued as defined by the broker, which will route this message to the right queue for the PDF creation task. The consumer then consumes this message, generates a PDF, and sends an email attaching the same PDF on behalf of the application.